# 5D Belief-State Adaptive Fractionation Solver

A Developer Tutorial for `adaptive_fractionation_overlap`

Internal repository design note

March 21, 2026

**Abstract**

This document is a *developer tutorial*: a teaching-oriented, in-depth walkthrough of the 5D belief-state adaptive fractionation solver used in the `adaptive_fractionation_overlap` package. Its main implementation lives in `core_adaptfx.py`, together with the supporting modules (`belief_model.py` and `helper_functions.py`). It is written in a tutorial style, with deliberate repetition across multiple abstraction levels, and is *not* optimised for skimming or quick lookup. Readers who want a concise implementation reference should read the code directly; this document is for those who want to understand *why* the code is structured the way it is.

The target reader is comfortable with Python but may not have a background in dynamic programming, Bayesian statistics, or NumPy's advanced features. Every mathematical concept is explained in plain language before its formal definition. Every non-trivial code path is reproduced verbatim alongside the equations it implements, with prose explaining what each step achieves. Familiarity with the clinical context (adaptive fractionation, overlap volumes, dose prescription) is assumed; this document explains the *algorithm*, not the Clinical Implementation. Readers who want the high-level design rationale and the plan for Stage B (full Normal-Inverse-Gamma belief state) should consult `DimensionalityExtensionPlan.tex` in the same directory.

## Contents

**1 High-Level Overview** **4**
- 1.1 What the algorithm does . . . 4
- 1.2 How the 5D Extension improves the baseline . . . 4
- 1.3 File map . . . 4

**2 Background Concepts** **5**
- 2.1 Dynamic programming — the core idea . . . 5
- 2.2 Probabilistic modelling — prior, posterior, and MAP . . . 7
- 2.3 Hypothetical branches: imagining future overlaps . . . 8
- 2.4 Running mean and variance: Welford's algorithm . . . 8
- 2.5 NumPy broadcasting: arrays of different shapes computed together . . . 9
- 2.6 NumPy advanced indexing: gathering values at irregular positions . . . 9

**3 Parameters, Constants, and Notation** **11**
- 3.1 Clinical parameters . . . 11
- 3.2 Penalty function constants . . . 11

**4 Helper Functions** **12**
4.1 `std_calc` — MAP estimate of the overlap standard deviation . . . 12
4.2 `get_state_space` — patient-specific overlap grid . . . 13
4.3 `probdist` — bin probabilities for an overlap distribution . . . 14
4.4 Penalty functions . . . 14
4.4.1 `penalty_calc_single` — penalty for one dose and one overlap . . . 15
4.4.2 `penalty_calc_matrix` — penalty for all dose–volume combinations at once . 15
4.5 Inline feasibility clipping — how much dose can we still deliver? . . . 16
4.6 `nearest_idx` — snapping continuous values to a grid . . . 16
4.7 `linear_interp` — interpolating between grid points . . . 17
4.8 `min_dose_to_deliver` — minimum dose needed right now . . . 17

**5 Belief-State Grids** **18**
5.1 The mu grid `_MU_GRID`: covering all patients . . . 18
5.2 The sigma grid `_SIGMA_GRID`: covering variability . . . 19
5.3 The fixed overlap state space `_VOLUME_SPACE` . . . 19
5.4 Precomputed branch probabilities `_P_BELIEF` . . . 20

**6 Belief Update: Welford Running Moments** **22**
6.1 The update equations . . . 22
6.2 Grid projection after the update . . . 22
6.3 Scalar belief update: `_hypothetical_belief_grid_indices` . . . 23
6.4 Expected future value for a scalar belief: `_bellman_expectation` . . . 23

**7 The Bellman Recursion: `adaptive_fractionation_core`** **25**
7.1 Two key arrays: `dose_space` and `action_space` . . . 25
7.2 State representation and array layout . . . 25
7.3 Action set . . . 26
7.4 Hard feasibility guards . . . 26
7.5 Initialisation from observed data . . . 27
7.6 The DP backward-pass loop . . . 27
7.7 Case 1: Terminal fraction (`i == 0`) . . . 28
7.8 Case 2: Hypothetical future fractions (`i $>$ 0`) . . . 29
7.9 Case 3: Actual fraction — first fraction ($f = 1$) . . . 31
7.10 Case 4: Actual fraction — intermediate ($f < F$, $f > 1$) . . . 32
7.11 Case 5: Actual fraction is the last fraction ($f = F$) . . . 33
7.12 Return values . . . 33

**8 Public API Functions** **34**
8.1 `adaptfx_full`: replay a complete treatment course . . . 34
8.2 `precompute_plan`: build a dose lookup table . . . 35

**9 End-to-End Call Flow** **37**

**10 Grid Sizes and Computational Cost** **38**
10.1 Grid dimensions at default settings . . . 38
10.2 Memory . . . 38
10.3 Operation count . . . 38

**11 Key Design Decisions and Their Rationale** **39**

# 1 High-Level Overview

## 1.1 What the algorithm does

At each fraction of treatment, the algorithm receives:

- all overlap volumes observed so far: $o_0, o_1, \ldots, o_f$ (cubic centimetres). $o_0$ is the overlap measured during the *planning scan* — an imaging session before treatment begins, from which the treatment plan is made. $o_1, \ldots, o_f$ are the overlaps measured at each delivered treatment fraction. The algorithm is *not* called at the planning scan (there is no dose to deliver then), but $o_0$ is always included in `volumes` from fraction 1 onwards. At fraction $f$, `volumes` therefore has $f + 1$ elements, and `volumes[-1]` is always the current fraction's overlap.
- the accumulated dose to PTV already delivered before fraction $f$: $D_f = \sum_{k=1}^{f-1} d_k$ (Gy); note that because $d_k$ is drawn from the discretised action grid, $D_f$ only takes values that are multiples of $\delta_d$ (the dose step size, defined in the next bullet);
- clinical bounds on the treatment (total fractions $F$, minimum dose per fraction $d_{\min}$, maximum dose to PTV per fraction $d_{\max}$, prescribed mean $\bar{d}$, dose step size $\delta_d$).

It returns one number: the recommended dose $d_f$ for the current fraction.

The algorithm's goal is to *plan all remaining fractions at once*, not just the current one, so that the chosen dose today is the best possible choice in light of uncertain future overlaps ("best" is defined by the penalty function in Section 4.4: the algorithm minimises the expected total PTV underdosage cost, defined in Section 4.4), while aiming to deliver the total prescribed dose $D_{\text{goal}} = F \cdot \bar{d}$ (total fractions times prescribed mean fraction dose, from the input list above) exactly when the problem is feasible within the action bounds (see Section 7.4 for the cases where exact delivery is not guaranteed).

## 1.2 How the 5D Extension improves the baseline

The baseline solver used a fixed probability distribution over future overlaps, estimated once from all observed data and never revised as the DP explored hypothetical futures. In essence, the 5D extension personalises the algorithm: it learns each patient's overlap pattern as fractions are delivered, and plans future doses based on that evolving picture.

The 5D extension adds a *running belief* $(\mu, \sigma)$ to the planning state. This belief tracks a running estimate of the patient's typical overlap and its variability. Crucially, when the dynamic program imagines a hypothetical future overlap $o'$, it also updates the belief as if $o'$ had actually been observed. This means each hypothetical future has a distinct view of the patient, leading to appropriately different future decisions.

## 1.3 File map

| File | Role |
|---|---|
| `core_adaptfx.py` | Main DP solver, Numba kernel, public API |
| `belief_model.py` | Belief grids, branch probabilities, Bellman operators |
| `helper_functions.py` | Mathematical building blocks |
| `constants.py` | Default numerical parameters |

# 2 Background Concepts

This section builds up all the ideas needed to understand the algorithm from first principles. Readers who are comfortable with dynamic programming and Bayesian inference may skim this section.

## 2.1 Dynamic programming — the core idea

Dynamic programming (DP) is a method for making a *sequence of decisions* optimally. The key insight is: instead of trying to think about all future decisions simultaneously (which is exponentially hard), we work *backwards* from the last decision to the first.

To make this concrete, think about a 5-fraction treatment:

1. Fraction 1: entering with accumulated dose $D = 0$; OAR–PTV overlap $o_1$ cc; we deliver $d_1$ to the PTV.
2. Fraction 2: entering with $D = d_1$; overlap $o_2$ cc; we deliver $d_2$.
3. Fraction 3: entering with $D = d_1 + d_2$; overlap $o_3$ cc; we deliver $d_3$.
4. Fraction 4: entering with $D = d_1 + d_2 + d_3$; overlap $o_4$ cc; we deliver $d_4$.
5. Fraction 5: entering with $D = d_1 + d_2 + d_3 + d_4$; overlap $o_5$ cc; we deliver $d_5 = D_{\text{goal}} - D$, the exact remaining dose needed to hit the prescription.

We cannot choose all five doses upfront because we do not yet know the overlaps for fractions 2–5. But we can ask: "*if* I am at fraction 4 with accumulated dose $D$ and overlap $o_4$, what is the best possible outcome from here?" DP answers exactly this question, for every possible $(D, o)$ combination.

**State.** A *state* is a complete description of the situation at a given point in time. For our algorithm, the state at fraction $t$ is:

$$\text{state}_t = (t,\ D,\ o,\ \mu,\ \sigma).$$

This captures *everything* the algorithm needs to make a good decision at fraction $t$: which fraction we are on ($t$), how much dose has been accumulated ($D$), what today's overlap is ($o$), and what the running belief about the patient's distribution looks like $(\mu, \sigma)$.

**Value function.** The *value function* of a state is the maximum total reward achievable from that state onwards, using optimal decisions. The *reward* at each fraction is the negative of the PTV underdosage penalty $-c(d, o)$: because the OAR's dose limit forces the shared PTV–OAR tissue to receive at most $d_{\min}$, any dose $d > d_{\min}$ comes at a cost proportional to the overlap volume $o$. Maximising accumulated reward is therefore equivalent to minimising total PTV underdosage over the full course of treatment. We write the value function as $V_t(D, o, \mu, \sigma)$.

Think of the value function as the answer to: "starting in this state, what is the best possible PTV coverage I can achieve from here to the end of treatment?"

**Policy.** A *policy* is the decision rule: given the current state, which dose should I deliver? Once we have the value function, the policy at any state is simply the action that achieves the value function maximum:

$$\pi_t(D, o, \mu, \sigma) = \arg\max_d [-c(d, o) + \text{expected future value after delivering } d] \,.$$

The cost function $c(d, o)$ is defined precisely in Section 4.4; informally, it captures the PTV underdosage incurred when OAR overlap prevents the full prescription from being delivered to the shared tissue. The Bellman equation below uses the same notation.

**The Bellman equation — value function defined recursively.** The core mathematical identity of dynamic programming is the *Bellman equation*. In words: "the value function at a state is the immediate reward of the best action, plus the expected value of the state I will be in afterwards." In symbols:

$$V_t(D, o, \mu, \sigma) = \max_{d \in \mathcal{A}(D)} \Big[ \underbrace{-c(d, o)}_{\text{immediate reward}} + \underbrace{\mathbb{E}_{o'}\big[V_{t+1}(D + d, o', \mu', \sigma')\big]}_{\text{expected future value}} \Big]. \tag{1}$$

Here $\mathcal{A}(D)$ is the set of deliverable doses to the PTV at the current treatment fraction (doses that neither undershoot nor overshoot the prescription target given the remaining fractions). The expectation $\mathbb{E}_{o'}$ averages over all possible next overlaps $o'$, weighted by their probability. The updated belief $(\mu', \sigma')$ depends on which $o'$ was hypothetically observed.

**Why go backwards?** To compute $V_t$ we need $V_{t+1}$. To compute $V_{t+1}$ we need $V_{t+2}$. This chain keeps going until we reach the *last fraction*, where the decision is forced (deliver exactly the remaining PTV prescription). So we start there and compute the value function at the final fraction $F$, $V_F$, directly, then work backwards: $V_{F-1}$ uses $V_F$, $V_{F-2}$ uses $V_{F-1}$, and so on until we reach $V_f$, which gives us the optimal dose for the current fraction.

This is why the loop in `adaptive_fractionation_core` (`core_adaptfx.py`) runs from `i=0` (corresponding to the last fraction $F$) up to the current fraction $f$:

```
# In _build_dp_context (core_adaptfx.py):
for i, fraction_number in enumerate(
        np.arange(number_of_fractions, fraction_index_today, -1)):
    observation_count = int(fraction_number)
    if i != 0:
        # Hypothetical future fractions: fill values[i]
        # for all belief-grid (mi, si) entries.
        ...
    else:
        # Terminal fraction (i == 0).
        ...
```

`np.arange(number_of_fractions, fraction_index_today, -1)` generates $[F, F-1, \ldots, f+1]$, so `i=0` always corresponds to fraction $F$ (the terminal fraction), and the loop covers only *future* fractions. The current fraction $f$ is resolved separately in `_resolve_current_fraction`, which calls `_bellman_expectation` with the actual starting belief.

**Reward vs. penalty sign convention.** The algorithm *maximises* cumulative reward. The reward for delivering dose $d$ with overlap $o$ is defined as the *negative* of the penalty: $-c(d, o)$. A large overlap with a high dose is heavily penalised (large $c$), which becomes a very negative reward, which the maximiser avoids. Equivalently, you can think of the algorithm as minimising total penalty — the sign flip is just a convention so that DP (which is stated as a maximisation) applies directly.

## 2.2 Probabilistic modelling — prior, posterior, and MAP

**What is a probability distribution over parameters?** We model each patient's overlaps as drawn from a Normal distribution $\mathcal{N}(\mu, \sigma^2)$, where $\mu$ is the patient's typical overlap and $\sigma$ is how variable it is across fractions. In reality, we do not know $\mu$ and $\sigma$; we have to *estimate* them from the observed overlaps.

**Prior: knowledge before seeing data.** Before observing anything about a specific patient, we have population-level knowledge: across many patients, the variability $\sigma$ tends to be small and positive, rarely exceeding a few cc. We encode this as a *prior distribution* over $\sigma$ — a probability distribution expressing our beliefs about $\sigma$ before we see any data. The algorithm uses a *Gamma distribution* as the prior:

$$\sigma \sim \text{Gamma}(\alpha, \beta),$$

where $\alpha$ and $\beta$ are fitted from historical patient data. The Gamma distribution is always positive (good, since $\sigma$ must be positive) and right-skewed (most patients have small variability; a few have large).

**Likelihood: what the data tells us.** After observing $n$ overlaps $o_1, \ldots, o_n$, we can ask: how likely is this data if the true standard deviation were $\sigma$? This is the *likelihood* $L(\sigma \mid o_{1:n})$.

**Posterior: combining prior and data.** Bayes' rule says the posterior distribution (our updated belief after seeing data) is proportional to the prior times the likelihood:

$$p(\sigma \mid o_{1:n}) \propto p(\sigma) \cdot L(\sigma \mid o_{1:n}).$$

In plain English: the posterior is our belief about $\sigma$ after combining what we knew before (the prior) with what the data told us (the likelihood).

**MAP: the single best estimate.** The *maximum a posteriori* (MAP) estimate is the value of $\sigma$ that makes the posterior largest — the mode of the posterior distribution. Formally,

$$\hat{\sigma}_{\text{MAP}} = \arg\max_{\sigma} \; p(\sigma \mid o_{1:n}).$$

This is a single number, not a full distribution. Using MAP gives us a principled single estimate that automatically incorporates both the data and the prior.

A key property of MAP: when $n = 1$ (only one overlap observed), the sample variance is zero, which would give $\hat{\sigma} = 0$ and break the model. The MAP estimate avoids this by pulling toward the prior when data is scarce.

### 2.3 Hypothetical branches: imagining future overlaps

At online re-planning time, we do not know what future overlaps will be. The DP handles this by *imagining every possible future overlap* and weighting outcomes by probability. Each imagined future is called a *branch*.

In the 5D extension, when the DP imagines a hypothetical next overlap $o'$, it does two things:

1. It computes the probability of $o'$ under the current belief $(\mu, \sigma)$.
2. It updates the belief as if $o'$ had been observed, producing a new belief $(\mu', \sigma')$ specific to that branch.

The second point is the core innovation of the 5D extension. Different hypothetical overlaps lead to different updated beliefs, and those different beliefs lead to different optimal decisions for subsequent fractions. A branch where all future overlaps are large will learn that this patient has high overlap, and plan conservatively. A branch where overlaps are small will learn the opposite. The benefit is largest for patients whose true intra-fraction variability deviates most from the population average: highly variable patients are identified earlier and dosed more conservatively, while consistent patients are identified and dosed more aggressively.

**How branches are handled computationally.** We discretise the possible overlap volumes into a finite grid of $J$ bins. For each belief $(\mu, \sigma)$, branch $j$ has probability $p_j$ (computed from the Normal distribution) and leads to an updated belief $(\mu'_j, \sigma'_j)$. The expected future value is then the weighted average:

$$\mathbb{E}_{o'}[V_{t+1}(\ldots)] = \sum_{j=0}^{J-1} p_j \cdot V_{t+1}(\ldots \text{at branch-}j \text{ belief} \ldots).$$

### 2.4 Running mean and variance: Welford's algorithm

To update the belief $(\mu, \sigma)$ when a new overlap $o'$ is added, we need a formula for the new running mean and standard deviation. The *Welford algorithm* does this without storing all previous observations.

Suppose we have seen $n$ values with mean $\mu$ and population standard deviation $\sigma$. When we add one more value $o'$:

1. Compute the new mean:
$$\mu' = \frac{n\,\mu + o'}{n+1}.$$
This is simply the weighted average of the old mean (weight $n$) and the new value (weight 1). Intuitively: as $n$ grows large, a single new value barely moves the mean.

2. Compute the new variance using the two-factor Welford formula:
$$(\sigma')^2 = \frac{n\,\sigma^2 + (o' - \mu)(o' - \mu')}{n+1}.$$
The term $(o' - \mu)(o' - \mu')$ measures how much the new value "surprises" the distribution. If $o'$ is close to the old mean, the variance barely changes. If $o'$ is far away, the variance grows.

The formula uses *population variance*; for a full derivation and worked examples see `https://changyaochen.github.io/welford/`.

The corresponding code, taken from `_hypothetical_belief_grid_indices` in `belief_model.py`:

```
mu_prime     = (observation_count * mu + o_prime) / (observation_count + 1)
sigma2_prime = (observation_count * sigma**2
                + (o_prime - mu) * (o_prime - mu_prime)) / (observation_count + 1)
sigma_prime  = np.sqrt(np.maximum(sigma2_prime, _SIGMA_MIN**2))
```

Here `o_prime` is the hypothetical next overlap (a scalar or array), `observation_count` is the current observation count, and `_SIGMA_MIN` is a hard floor that prevents degenerate zero-variance beliefs.

## 2.5 NumPy broadcasting: arrays of different shapes computed together

Several places in the code perform calculations over three-dimensional grids of $(\mu_i, \sigma_s, o_j)$ values at once. NumPy achieves this with *broadcasting*: arrays with compatible shapes are automatically expanded to match each other without copying data.

**Example 2.1** (Broadcasting over a 2-D grid)**.** Suppose we want to compute $\mu_i + o_j$ for every pair of values from two arrays $\mu = [1, 2, 3]$ (shape $(3, )$) and $o = [10, 20]$ (shape $(2, )$). We reshape $\mu$ to shape $(3, 1)$ and $o$ to shape $(1, 2)$; NumPy then produces a $(3, 2)$ result automatically:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 10 & 20 \end{pmatrix} = \begin{pmatrix} 11 & 21 \\ 12 & 22 \\ 13 & 23 \end{pmatrix}.$$

In Python: `mu[:, None] + o[None, :]` gives exactly this. The `None` inserts a length-1 axis, telling NumPy which dimensions to broadcast over.

In the code, three-dimensional broadcasting like `mu[:, None, None] + o[None, None, :]` produces a $(N_\mu, 1, J)$-shaped result that NumPy expands to $(N_\mu, N_\sigma, J)$ when combined with a $(1, N_\sigma, 1)$-shaped sigma array. This computes the Welford update for *all* $N_\mu \times N_\sigma \times J$ combinations at once in a single vectorised pass. The relevant lines from `belief_model.py` are:

```
mu_vals    = _MU_GRID[:, None, None]       # shape (N_mu, 1, 1)
sigma_vals = _SIGMA_GRID[None, :, None]    # shape (1, N_sigma, 1)
o_vals     = _VOLUME_SPACE[None, None, :]  # shape (1, 1, N_overlap)

mu_prime     = (observation_count * mu_vals + o_vals) / (observation_count + 1)
delta        = o_vals - mu_vals
delta_prime  = o_vals - mu_prime
sigma2_prime = (observation_count * sigma_vals**2 + delta * delta_prime) / (observation_count +
```

Each assignment produces a full $(N_\mu, N_\sigma, N_{\text{overlap}})$ array via broadcasting, with no explicit Python loops over the grid axes.

## 2.6 NumPy advanced indexing: gathering values at irregular positions

Standard NumPy indexing selects a rectangular sub-array (e.g. `A[2:5, 0:3]`). *Advanced indexing* (also called fancy indexing) uses an array of indices to gather values at arbitrary, non-contiguous positions.

**Example 2.2** (Fancy indexing)**.** If `A = [10, 20, 30, 40, 50]` and `idx = [0, 3, 1]`, then `A[idx]` gives `[10, 40, 20]`. The index array can have any shape, and the output shape matches.

The algorithm uses this to look up, for each overlap branch $j$, the value stored at the *updated* belief indices ($\texttt{next_mi}[j], \texttt{next_si}[j]$), where different $j$'s point to different positions in the value table. A plain slice cannot express this because the positions are irregular: each branch $j$ corresponds to a different hypothetical overlap $o_j$, which after the Welford update lands on a different $(\mu', \sigma')$ grid cell. The resulting index arrays `next_mi` and `next_si` are in general non-contiguous — branch 0 might land on grid cell $(3, 1)$ while branch 1 lands on $(3, 2)$ and branch 2 on $(5, 1)$ — so there is no rectangular slice that selects exactly these cells.

# 3 Parameters, Constants, and Notation

## 3.1 Clinical parameters

| Python name | Symbol | Meaning |
|---|---|---|
| `number_of_fractions` | $F$ | Total fractions in the treatment course |
| `min_dose` | $d_{\min}$ | Minimum allowed fraction dose (Gy) |
| `max_dose` | $d_{\max}$ | Maximum allowed fraction dose (Gy) |
| `mean_dose` | $\bar{d}$ | Prescribed mean fraction dose (Gy) |
| `dose_steps` | $\delta_d$ | Dose grid step size (Gy) |
| `alpha`, `beta` | $\alpha, \beta$ | Gamma prior shape and scale on $\sigma$ (standard deviation) |
| `accumulated_dose` | $D_f$ | Total dose delivered before fraction $f$ |
| `volumes` | $o_{0:f}$ | Planning scan + all treatment-fraction overlaps so far (cc); `volumes[-1]` is |

Default values (from `constants.py`):

```
DEFAULT_MIN_DOSE  = 6.0
DEFAULT_MAX_DOSE  = 10.0
DEFAULT_MEAN_DOSE = 8.0
DEFAULT_DOSE_STEPS = 0.5
DEFAULT_NUMBER_OF_FRACTIONS = 5
DEFAULT_ALPHA = 1.072846744379587
DEFAULT_BETA  = 0.7788684130749829
```

The prescribed goal dose is

$$D_{\text{goal}} = F \cdot \bar{d}. \tag{2}$$

For the defaults this is $5 \times 8 = 40$ Gy. Hard feasibility: the treatment is considered infeasible if $D_{\text{goal}}$ cannot be reached within the action bounds (see Section 7.4).

## 3.2 Penalty function constants

Two constants govern the shape of the per-fraction cost (from `constants.py`):

```
SLOPE     = -0.65   # penalty steepness per cc of overlap
INTERCEPT = 0.0
```

Their role is explained in Section 4.4, where the full penalty formula is derived.

# 4 Helper Functions

This section documents each function in `helper_functions.py`. For each function we explain the purpose in plain language, give the mathematical formula, and show the exact code.

## 4.1 `std_calc` — MAP estimate of the overlap standard deviation

**Purpose.** Given $n$ observed overlap volumes and the Gamma prior hyperparameters, this function returns the single best estimate of the patient's overlap standard deviation $\sigma$. "Best" here means the MAP estimate: the value of $\sigma$ that makes the posterior probability $p(\sigma \mid o_{1:n})$ largest.

**Why not just use the sample standard deviation?** With only 1 observation the sample standard deviation is zero (or undefined with Bessel's correction), which would make the model degenerate. The MAP estimate avoids this by incorporating the Gamma prior, which pulls the estimate toward a plausible non-zero value when data is scarce.

**Mathematical derivation.** We model overlaps as $o_i \sim \mathcal{N}(\mu, \sigma^2)$, with the unknown mean $\mu$ replaced by the sample mean $\bar{o}$. Per the 2025 paper (equation (5) and Figure 3), the prior is placed on the standard deviation:

$$\sigma \sim \text{Gamma}(\alpha, \beta).$$

**Step 1: Bayes in log form.** By Bayes' theorem, $p(\sigma \mid o_{1:n}) \propto p(\sigma) \cdot L(\sigma \mid o_{1:n})$. Taking logarithms (which does not change where the maximum is):

$$\log p(\sigma \mid o_{1:n}) = \underbrace{\log p(\sigma)}_{\text{log-prior}} + \underbrace{\log L(\sigma \mid o_{1:n})}_{\text{log-likelihood}} + \text{const.}$$

**Step 2: log-prior.** For $\sigma \sim \text{Gamma}(\alpha, \beta)$:

$$\log p(\sigma) = (\alpha - 1) \ln \sigma - \frac{\sigma}{\beta} + \text{const.}$$

**Step 3: log-likelihood.** With $\mu$ replaced by the sample mean (profile likelihood), the Normal log-likelihood for $n$ observations with sample variance $s^2$ is:

$$\log L(\sigma \mid o_{1:n}) = -(n-1) \ln \sigma - \frac{ns^2}{2\sigma^2} + \text{const.}$$

The exponent is $n - 1$ rather than $n$ because estimating $\mu$ from the data consumes one degree of freedom (the same reason sample variance divides by $n - 1$, Bessel's correction).

**Step 4: combine.** Adding the two terms and dropping constants gives the log-posterior $\ell(\sigma)$:

$$\ell(\sigma) = (\alpha - 1) \ln \sigma - (n-1) \ln \sigma - \frac{\sigma}{\beta} - \frac{s^2}{2\,\sigma^2/n}, \tag{3}$$

where $s^2 = \text{Var}(o_{0:n-1})$ is the population variance of all $n$ observed overlaps (including the planning scan). Each term has a clear origin:

- $(\alpha - 1) \ln \sigma$: log-prior shape term;
- $-(n-1) \ln \sigma$: log-likelihood normalisation, with one degree of freedom used for estimating $\mu$;

- $-\sigma/\beta$: log-prior scale term;
- $-s^2/(2\sigma^2/n)$: log-likelihood data-fit term.

**Step 5: from log-posterior to code.** The MAP estimate is $\arg\max_\sigma \ell(\sigma)$. Because the exponential function is monotone, maximising $\ell(\sigma)$ is identical to maximising $\exp(\ell(\sigma))$ — the un-normalised posterior. The code exploits this: instead of differentiating $\ell(\sigma)$ analytically (which yields a cubic equation with no clean closed form), it evaluates $\exp(\ell(\sigma))$ on a fine grid and takes the argmax. The variable `likelihood_values` in the snippet below is $\exp(\ell(\sigma))$, not $\ell(\sigma)$ itself:

```
def std_calc(measured_data, alpha, beta):
    n = len(measured_data)
    std_values = np.arange(0.001, 10, 0.001)
    measured_variance = np.var(measured_data)
    likelihood_values = (
        std_values ** (alpha - 1)
        / std_values ** (n - 1)
        * np.exp(-1 / beta * std_values)
        * np.exp(-measured_variance / (2 * (std_values**2 / n)))
    )
    std = std_values[np.argmax(likelihood_values)]
    return std
```

The expression `likelihood_values` is exactly $\exp(\ell(\sigma))$:

- `std_values**(alpha-1)`: $\sigma^{\alpha-1}$, the prior shape term.
- `/ std_values**(n-1)`: $\sigma^{-(n-1)}$, the likelihood normalisation.
- `np.exp(-1/beta * std_values)`: $e^{-\sigma/\beta}$, the prior rate term.
- `np.exp(-measured_variance / (2 * std_values**2 / n))`: the data fit term.

**Note 4.1.** The grid scan over 9,999 values might seem crude, but the MAP of a log-concave posterior is smooth and the step size 0.001 cc gives more than adequate precision for a clinical dose planner.

### 4.2 `get_state_space` — patient-specific overlap grid

**Purpose.** Given a Normal distribution $\mathcal{N}(\mu, \sigma^2)$, construct an e.g. 200-point grid spanning the interval that contains 99.8% of the distribution's probability mass. This grid is used for display purposes and for the `precompute_plan` scanning loop (`precompute_plan` sweeps over a range of possible current-fraction overlap values and returns the dose the algorithm would recommend for each, producing a dose–overlap response curve for visualisation); it is *not* used as the DP's overlap state space (see Section 5.3 for why).

```
def get_state_space(distribution):
    mean_volume, std_volume = distribution
    lower_bound = norm.ppf(0.001, loc=mean_volume, scale=std_volume)
    upper_bound = norm.ppf(0.999, loc=mean_volume, scale=std_volume)
    return np.linspace(lower_bound, upper_bound, 200)
```

`norm.ppf(0.001, ...)` is the 0.1th percentile of the Normal distribution (the inverse CDF, also called the percent-point function). Here $\Phi$ denotes the standard Normal CDF and $\Phi^{-1}$ its inverse (the quantile function); the resulting grid spans $[\Phi^{-1}(0.001; \mu, \sigma),\ \Phi^{-1}(0.999; \mu, \sigma)]$ with equally spaced points.

### 4.3 `probdist` — bin probabilities for an overlap distribution

**Purpose.** Given a Normal distribution and a discrete grid of overlap values, compute the probability that the next overlap falls in each grid bin. Think of each grid point as the centre of a bin of width $h$; this function asks: "what fraction of the distribution's mass lands in each bin?"

**Formula.** For a grid with uniform spacing $h$ and bin centres $c_0, c_1, \ldots$, the probability of landing in bin $j$ is the area under the Normal density between $c_j - h/2$ and $c_j + h/2$:

$$p_j = \Phi\left(\frac{c_j + h/2 - \mu}{\sigma}\right) - \Phi\left(\frac{c_j - h/2 - \mu}{\sigma}\right), \tag{4}$$

where $\Phi$ is the standard Normal CDF.

```
def probdist(X, state_space):
    spacing = state_space[1] - state_space[0]
    upper_bounds = state_space + spacing / 2
    lower_bounds = state_space - spacing / 2
    mean_volume, std_volume = X
    prob = norm.cdf(upper_bounds, loc=mean_volume, scale=std_volume) \
         - norm.cdf(lower_bounds, loc=mean_volume, scale=std_volume)
    return prob
```

**Note 4.2.** This function does *not* assign left/right tail probability to the edge bins. If the distribution extends beyond the grid, the probabilities will sum to slightly less than 1. This is acceptable for the initial-belief probability vector (`current_overlap_probs`) because `_VOLUME_SPACE` is wide enough to cover any realistic patient. The belief-grid branch probabilities `_P_BELIEF` handle tails explicitly (Section 5.4).

### 4.4 Penalty functions

**Purpose.** These functions compute the "cost" of delivering a given dose when the overlap volume is a given value. The penalty is zero when the dose equals the minimum ($d = d_{\min}$) and grows as dose rises above $d_{\min}$.

Intuitively: $d_{\min}$ is the OAR dose limit. At $d = d_{\min}$, the dose delivered to the PTV matches this limit exactly — the OAR's constraint is fully met and the PTV in the overlap region receives its prescribed dose without underdosage. The penalty only accrues for $d > d_{\min}$: delivering more than $d_{\min}$ to the PTV means the overlap tissue, which is capped at $d_{\min}$ by the OAR constraint, is underdosed by $(d - d_{\min})$ Gy per cc of overlap. The growth is faster when the overlap is larger, since more shared tissue is affected by each additional Gy above $d_{\min}$.

**Mathematical form.** The penalty for delivering dose $d$ with overlap $o$ is:

$$c(d,\, o) = \begin{cases} 0 & d < d_{\min}, \\ (d - d_{\min})\, o \;+\; \dfrac{(d - d_{\min})^2}{2}\, |\,\mathrm{INTERCEPT} + \mathrm{SLOPE} \cdot o\,| & d \geq d_{\min}. \end{cases} \tag{5}$$

With the default constants (INTERCEPT $= 0$, SLOPE $= -0.65$):

$$c(d,\, o) = (d - d_{\min})\, o \;+\; \frac{(d - d_{\min})^2}{2} \cdot 0.65\, o. \tag{6}$$

The penalty has two parts:

- A *linear* part $(d - d_{\min}) \cdot o$: every Gy above the minimum incurs a cost proportional to the overlap.
- A *quadratic* part $\frac{(d-d_{\min})^2}{2} \cdot 0.65\, o$: an additional cost that grows faster than linearly, making large excesses increasingly expensive.

When $o = 0$, both terms are zero: no OAR tissue, no cost. When $o$ is large, even a small excess above $d_{\min}$ is costly.

### 4.4.1 `penalty_calc_single` — penalty for one dose and one overlap

This function handles scalar inputs or small arrays.

**`np.where` — element-wise conditional selection.** NumPy's `np.where(condition, x, y)` returns an array of the same shape as `condition`: wherever `condition` is `True` it takes the corresponding value from `x`; wherever it is `False` it takes from `y`. For example, `np.where([True, False, True], [1,2,3], [10,20,30])` gives `[1,20,3]`. It is the vectorised equivalent of writing `x if condition else y` element by element.

The `np.where` at the end sets the penalty to zero wherever `physical_dose < min_dose`, implementing the piece-wise definition (5).

```
def penalty_calc_single(physical_dose, min_dose, actual_volume,
                        intercept=INTERCEPT, slope=SLOPE):
    physical_dose = np.asarray(physical_dose)
    min_dose = np.asarray(min_dose)
    steepness = np.abs(intercept + slope * actual_volume)
    penalty_added = (physical_dose - min_dose) * (actual_volume) \
                  + (physical_dose - min_dose)**2 * steepness / 2
    penalty_added = np.where(physical_dose < min_dose, 0, penalty_added)
    if np.isscalar(penalty_added) or penalty_added.shape == ():
        return float(penalty_added)
    return penalty_added
```

### 4.4.2 `penalty_calc_matrix` — penalty for all dose–volume combinations at once

Inside the DP loop, we need the penalty $c(d_k, o_j)$ for every combination of action $d_k$ and overlap $o_j$. Computing these one at a time would be slow; `penalty_calc_matrix` vectorises the calculation using `np.outer`, which computes the outer product of two vectors.

If `volume_space` has $J$ entries and `action_space` has $A$ entries, `np.outer(volume_space, action_space - min_dose)` produces a $J \times A$ matrix whose $(j, k)$ entry is $o_j \cdot (d_k - d_{\min})$.

```
def penalty_calc_matrix(delivered_doses, volume_space, min_dose,
                        intercept=INTERCEPT, slope=SLOPE):
    steepness = np.abs(intercept + slope * volume_space)
    overlap_penalty_linear    = np.outer(volume_space,
                                         (delivered_doses - min_dose))
    overlap_penalty_quadratic = np.outer(steepness,
                                         (delivered_doses - min_dose)**2) / 2
    overlap_penalty = overlap_penalty_linear + overlap_penalty_quadratic
    return overlap_penalty
```

The result is a matrix of shape $(J, A)$ where entry $(j, k)$ is $c(d_k, o_j)$. This is precomputed once per DP step and reused.

### 4.5 Inline feasibility clipping — how much dose can we still deliver?

**Purpose.** Given accumulated dose $D$ and goal $D_{\text{goal}}$, the highest dose we can deliver at this fraction without immediately overshooting the prescription is:

$$d_{\max}^{\text{feas}}(D) = \min\bigl(d_{\max},\ D_{\text{goal}} - D\bigr). \tag{7}$$

This clipping is now performed inline inside `adaptive_fractionation_core` rather than by a separate helper function. For the intermediate fraction case (Section 7.10), the largest feasible action index is computed as:

```
min_dose_reserved_for_future = (number_of_fractions - fraction_index_today) * min_dose
max_allowed_action = np.minimum(action_space[-1],
                                prescribed_dose - accumulated_dose
                                - min_dose_reserved_for_future)
max_action_index = np.abs(action_space - max_allowed_action).argmin()
max_action_index = max(max_action_index, 1)
feasible_actions = action_space[:max_action_index + 1]
```

The guard `max_action_index = max(..., 1)` ensures that at least the minimum dose (`action_space[0]`) is always available even if arithmetic would otherwise exclude it. `min_dose_reserved_for_future` is the dose budget that must be kept for all fractions *after* the current one; subtracting it gives the tightest single-fraction ceiling.

### 4.6 `nearest_idx` — snapping continuous values to a grid

**Purpose.** The DP's belief state $(\mu, \sigma)$ is continuous, but the value table only stores values at the finite grid points in `_MU_GRID` and `_SIGMA_GRID`. After the Welford update produces continuous $\mu'$ and $\sigma'$, we must snap each to the nearest grid point. This function does that efficiently for large arrays.

```
def nearest_idx(values, grid):
    flat = np.asarray(values).ravel()
    hi = np.searchsorted(grid, flat, side='left').clip(1, len(grid) - 1)
    lo = hi - 1
    idx = np.where(flat - grid[lo] <= grid[hi] - flat, lo, hi)
    return idx.reshape(np.asarray(values).shape)
```

**How it works.** `np.searchsorted(grid, v)` finds, for each query $v$, the index $h$ such that `grid[h-1]` $\leq v <$ `grid[h]`. The nearest grid point is therefore either `grid[h-1]` or `grid[h]`, whichever is closer. The `np.where` picks the closer one.

**Why not use `np.argmin(np.abs(values - grid))`?** That approach has complexity $O(n \cdot K)$ where $n$ is the number of queries and $K$ is the grid size. `searchsorted` uses binary search, giving $O(n \log K)$. For the $K = 150$-point mu grid, this is roughly $20\times$ faster on large arrays — a meaningful saving since this function is called millions of times during the DP.

### 4.7 `linear_interp` — interpolating between grid points

**Purpose.** When looking up a future value at accumulated dose $D + d$, the query dose may not fall exactly on a dose-grid point. Linear interpolation estimates the value between the two nearest grid points.

```
def linear_interp(x_values, y_values, query_points):
    query = np.asarray(query_points)
    return np.interp(query.ravel(), x_values, y_values).reshape(query.shape)
```

This is a thin wrapper around `numpy.interp` that handles queries of any shape. `numpy.interp` does piecewise-linear interpolation: for a query $q$ between grid points $x_\ell$ and $x_h$, it returns $y_\ell + (q - x_\ell) \cdot (y_h - y_\ell)/(x_h - x_\ell)$.

### 4.8 `min_dose_to_deliver` — minimum dose needed right now

**Purpose.** Even if we could deliver any dose from $d_{\min}$ to $d_{\max}$, we must also ensure that the remaining fractions can still reach $D_{\text{goal}}$. If we deliver too little now, the remaining fractions (even all at $d_{\max}$) may not make up the shortfall. This function computes the minimum dose that must be delivered at the current fraction to keep the prescription reachable:

$$d_{\min}^{\text{feas}}(D, L) = \max\Big(d_{\min},\ D_{\text{goal}} - D - (L-1) \cdot d_{\max}\Big). \tag{8}$$

Here $L$ is the number of fractions remaining (including the current one). The expression $D_{\text{goal}} - D - (L-1) \cdot d_{\max}$ is "how much we need minus the maximum the remaining fractions can deliver" — if this is positive, we must deliver at least that much now.

```
def min_dose_to_deliver(accumulated_dose, fractions_left,
                        prescribed_dose, min_dose, max_dose):
    min_dose_to_deliver_calculated = (
        (prescribed_dose - accumulated_dose)
        - ((fractions_left - 1) * max_dose)
    )
    return (min_dose if min_dose_to_deliver_calculated < min_dose
            else min_dose_to_deliver_calculated)
```

This is used by `precompute_plan` (Section 8.2) to determine when the scanning loop can stop.

# 5 Belief-State Grids

The 5D extension's central innovation is carrying a belief $(\mu, \sigma)$ in the DP state. To store values and policies for every possible belief, $\mu$ and $\sigma$ are discretised to finite grids. This section describes those grids and explains every design choice.

**Note 5.1.** The specific grid values described in this section reflect the *current implementation*, benchmarked on a cohort of 58 patients. They should be treated as a reasonable starting point, not a final optimised design. Principled selection of grid resolution along all dimensions — $\mu$, $\sigma$, and the overlap state space — requires systematic numerical analysis (e.g. convergence studies and cross-validation on a larger patient population) to ensure the discretisation is neither too coarse (introducing approximation error) nor overfitted to the characteristics of the 58-patient cohort. This work is ongoing.

## 5.1 The mu grid `_MU_GRID`: covering all patients

The mu grid must span the full range of plausible patient mean overlaps. A *non-uniform* grid is used: fine resolution where most patients fall (small overlaps, $< 4$ cc) and coarser resolution for rare extreme cases:

```
_MU_GRID = np.unique(np.concatenate([
    np.linspace(0.0,  1.0,  38),   # step ~0.027 cc  -- typical range
    np.linspace(1.05, 4.0,  39),   # step ~0.078 cc
    np.linspace(4.1,  10.0, 42),   # step ~0.144 cc
    np.linspace(10.2, 16.0, 15),   # step ~0.414 cc
    np.linspace(16.5, 30.0, 16),   # step ~0.900 cc  -- extreme patients
]))  # 150 grid points total
```

| Segment | Range (cc) | Points | Step (cc) |
|---|---|---|---|
| 1 | $[0.00, 1.00]$ | 38 | $\approx 0.027$ |
| 2 | $[1.05, 4.00]$ | 39 | $\approx 0.078$ |
| 3 | $[4.10, 10.00]$ | 42 | $\approx 0.144$ |
| 4 | $[10.20, 16.00]$ | 15 | $\approx 0.414$ |
| 5 | $[16.50, 30.00]$ | 16 | $\approx 0.900$ |
| **Total** | $[0, 30]$ | **150** | — |

**Why non-uniform?** A coarse uniform grid (e.g. step 1.25 cc) would map a patient with true mean overlap $\mu = 0.37$ cc to the nearest grid point $\mu = 0$ cc. Now consider what happens in the DP: a $\mathcal{N}(0, \sigma^2)$ distribution is centred at exactly zero. Since `_VOLUME_SPACE` starts at 0 cc, every branch from this belief lands at the leftmost bin. The DP sees all future overlaps as 0 cc and assigns zero penalty to everything — completely wrong. Fine resolution near 0 cc prevents this pathology.

**Why extend to 30 cc?** Patients with observed mean overlap $> 12$ cc exist in the 58-patient cohort. If the grid ended at 12 cc, those patients' beliefs would clip to the last grid point, causing systematic underestimation of their overlap distribution and therefore overly aggressive dosing.

**Why 150 points total?** A sweep over $N_\mu$ from 50 to 500 (on the 58-patient cohort) shows that benefit quality saturates at approximately $N_\mu = 150$: above that, the improvement fluctuates within $\pm 0.005$ ccGy — the level of quantisation noise rather than a genuine resolution benefit. $N_\mu = 150$ captures 99.95% of the benefit achievable at $N_\mu = 500$ at $2.3\times$ lower compute. Earlier versions used $N_\mu = 280$; the reduction was made after this sweep.

### 5.2 The sigma grid `_SIGMA_GRID`: covering variability

The sigma grid spans the range of plausible per-patient overlap variabilities. Each $\sigma$ value represents how consistently a patient's overlap volume remains from fraction to fraction: a small $\sigma$ means the overlap is nearly the same every day; a large $\sigma$ means it fluctuates substantially. A non-uniform grid is used, with denser coverage at low variabilities (where most patients fall) and sparser coverage at the high-variability tail:

```
_SIGMA_GRID = np.unique(np.concatenate([
    np.linspace(0.05, 0.7, 5),    # step ~0.163 cc  -- most patients
    np.linspace(0.8,  1.8, 3),    # step ~0.500 cc  -- upper range
    np.linspace(2.0,  4.5, 3),    # step  1.250 cc  -- high variability tail
]))  # 11 grid points total
_SIGMA_MIN = float(_SIGMA_GRID[0])   # 0.05 cc -- hard floor on sigma
```

The upper limit of 4.5 cc is set to avoid clipping the most variable patient in the 58-patient ACTION cohort: patient 3 has MAP $\hat{\sigma} = 4.05$ cc across all observed fractions. Extending the tail from the previous 3.5 cc to 4.5 cc required adding one point to the third segment (2 points $\to$ 3, giving the total 10 $\to$ 11).

The floor $\sigma_{\min} = 0.05$ cc serves two purposes:

1. It prevents division-by-zero in the Normal density calculation (a distribution with $\sigma = 0$ is a point mass, not a proper distribution).

2. It ensures that even a patient whose overlaps are very consistent still has a non-degenerate predictive distribution, so the DP continues to spread probability over multiple future branches.

### 5.3 The fixed overlap state space `_VOLUME_SPACE`

This is the fixed, uniform grid of overlap volumes over which the DP operates. Unlike the $\mu$ and $\sigma$ grids, it is shared by every patient and every fraction — a single wide grid covering all clinically realistic overlap values. Each of its $J$ bins represents one possible overlap volume outcome that the DP considers when imagining hypothetical future fractions.

```
_VOLUME_SPACE = np.linspace(0.0, 44.0, 441)
```

This produces a 441-point uniform grid from 0 cc to 44 cc, with spacing exactly 0.10 cc (matching TPS output resolution). The upper bound is hardcoded rather than derived from `_SIGMA_GRID[-1]`, so that the bin width stays exactly 0.1 cc regardless of future $\sigma$ grid changes. The value 44 cc comfortably covers $\mu_{\max} + 3\sigma_{\max} = 30 + 3 \times 4.5 = 43.5$ cc; the tail probability beyond 44 cc is negligible for any belief on the grid.

**Key Idea 5.1** (Why a fixed wide grid is critical — and what goes wrong without it)**.** A tempting design choice is to use a patient-specific overlap grid (from `get_state_space`) as the DP's overlap

state space. This seems efficient: why model overlaps far from the patient's observed range? But it causes a serious bug.

Consider a patient on fraction 1 whose only observed overlap is 19.79 cc. The patient-specific grid spans roughly $[19.11, 20.47]$ cc — a 1.36 cc window.

The DP must now compute transition probabilities for *every* belief grid point, including $\mu = 0.4$ cc. A $\mathcal{N}(0.4, \sigma^2)$ distribution has essentially all of its mass below 3 cc. But the patient-specific grid starts at 19.11 cc. The nearest available bin is 19.11 cc. So the DP assigns the entire probability mass of the $\mu = 0.4$ cc belief to the 19.11 cc bin — and plans as if this patient will have 19.11 cc overlaps forever, regardless of belief. The value function is completely corrupted for low-$\mu$ beliefs.

The fixed wide `_VOLUME_SPACE` spanning $[0, 44]$ cc ensures that for any $(\mu, \sigma)$ on the belief grid, the branch probabilities are correctly distributed and sum to 1. Using a patient-specific or belief-specific overlap range is explicitly prohibited for this reason.

### 5.4 Precomputed branch probabilities `_P_BELIEF`

For every belief grid point $(\mu_i, \sigma_s)$ and every overlap bin $j$, we need the probability of landing in bin $j$. This is exactly equation (4) applied to each $(\mu_i, \sigma_s)$ pair. Since the inputs (`_MU_GRID`, `_SIGMA_GRID`, `_VOLUME_SPACE`) are all fixed constants, we precompute the full $(N_\mu \times N_\sigma \times J)$ array *once at module import* and reuse it at every planning call.

**Tail handling.** *Tail handling* is the process of ensuring that probability mass falling outside the finite bounds of `_VOLUME_SPACE` is not lost. Because the Normal distribution has infinite support, any belief $(\mu, \sigma)$ will in principle assign some probability to overlaps below `_VOLUME_SPACE[0]` (the left tail) and above `_VOLUME_SPACE[-1]` (the right tail). Without correction, those out-of-bounds probabilities simply vanish, causing each row of `_P_BELIEF` to sum to less than 1. The DP would then systematically underestimate future values for beliefs near the grid edges.

The fix is to clamp: we assign the entire left-tail mass to the first bin (bin 0) and the entire right-tail mass to the last bin (bin $J-1$). Concretely, for beliefs with small $\mu$ (e.g. $\mu_0 = 0$ cc), most of the Normal distribution's mass falls below the grid — it is all folded into bin 0. Formally:

$$P_{i,s,0} \mathrel{+}= \Phi\left(\frac{c_0 - h/2 - \mu_i}{\sigma_s}\right), \tag{9}$$

$$P_{i,s,J-1} \mathrel{+}= 1 - \Phi\left(\frac{c_{J-1} + h/2 - \mu_i}{\sigma_s}\right). \tag{10}$$

After this adjustment, every row sums to exactly 1.

```
def _precompute_branch_probabilities():
    spacing = _VOLUME_SPACE[1] - _VOLUME_SPACE[0]
    upper_bounds = _VOLUME_SPACE + spacing / 2   # upper edge of each bin
    lower_bounds = _VOLUME_SPACE - spacing / 2   # lower edge of each bin
    # Core bin probabilities using broadcasting over (N_mu, N_sigma, N_overlap)
    probabilities = (
        norm.cdf(upper_bounds[None, None, :],
                 loc=_MU_GRID[:, None, None],
                 scale=_SIGMA_GRID[None, :, None])
      - norm.cdf(lower_bounds[None, None, :],
                 loc=_MU_GRID[:, None, None],
                 scale=_SIGMA_GRID[None, :, None])
```

```
    )
    # Add left-tail mass to bin 0
    probabilities[:, :, 0]  += norm.cdf(lower_bounds[0],
                               loc=_MU_GRID[:, None],
                               scale=_SIGMA_GRID[None, :])
    # Add right-tail mass to last bin
    probabilities[:, :, -1] += 1.0 - norm.cdf(upper_bounds[-1],
                                     loc=_MU_GRID[:, None],
                                     scale=_SIGMA_GRID[None, :])
    return probabilities

_P_BELIEF = _precompute_branch_probabilities()
# shape: (N_mu, N_sigma, N_overlap), computed once at module load
```

**Reading the broadcasting.** `upper_bounds[None, None, :]` has shape $(1, 1, J)$, `_MU_GRID[:, None, None]` has shape $(N_\mu, 1, 1)$, and `_SIGMA_GRID[None, :, None]` has shape $(1, N_\sigma, 1)$. NumPy broadcasts these to $(N_\mu, N_\sigma, J)$, computing all $150 \times 11 \times 441 = 727{,}650$ probabilities in one call.

`_P_BELIEF` uses approximately $150 \times 11 \times 441 \times 8 \approx 6$ MB of memory and is computed once at module load time.

# 6 Belief Update: Welford Running Moments

When the DP imagines a hypothetical next overlap $o'$, it must update the current belief $(\mu, \sigma)$ to reflect what the running estimate would look like after observing $o'$. This is the Welford update introduced in Section 2.4, now written out precisely.

## 6.1 The update equations

Suppose before fraction $t$ we have $n_t$ observations with running mean $\mu_t$ and running population variance $\sigma_t^2$. Observing (or hypothetically observing) $o'$ gives:

$$\mu' = \frac{n_t\,\mu_t + o'}{n_t + 1}, \tag{11}$$

$$(\sigma')^2 = \frac{n_t\,\sigma_t^2 + (o' - \mu_t)(o' - \mu')}{n_t + 1}. \tag{12}$$

The update count $n_t$ equals the fraction index: at fraction $f$, $n_t = f$. Inside the DP, for each hypothetical future fraction, the loop variable `observation_count` is set to `fraction_number` — the absolute fraction index being modelled. This represents the total expected count of observations at that hypothetical step of the backward pass (real observations already made plus hypothetical observations accumulated along the branch being explored), *not* solely a count of real-world observations. For example, when planning at fraction 2 and modelling fraction 4 hypothetically, `observation_count = 4`: as if four treatment-fraction observations have been accumulated along that branch.

**Why is $n_t = f$ rather than $f + 1$, even though `volumes` has $f + 1$ elements?** The planning scan $o_0$ is *not* counted in $n_t$. The count tracks treatment-fraction observations only $(o_1, \ldots, o_f)$. The planning scan is included in `volumes` and therefore in `initial_belief_mu` and `initial_belief_sigma`—it informs the initial belief—but it does not contribute a count to the Welford denominator. The reason is that $o_0$ is acquired before treatment begins, potentially under different imaging and positioning conditions than the daily treatment fractions. Assigning it a full count would over-weight that single pre-treatment measurement in the running estimate of precision. Instead, $o_0$ seeds the belief value of $\mu$; subsequent treatment fractions drive the count.

Concretely, at fraction 2 with `volumes` $= [o_0, o_1, o_2]$:

| | |
|---|---|
| `initial_belief_mu` | $= (o_0 + o_1 + o_2)/3$ (all three values) |
| `observation_count` for DP at fraction 2 | $= 2$ (treatment fractions only) |
| `observation_count` for DP at hypothetical fraction 3 | $= 3$ |
| `observation_count` for DP at hypothetical fraction 4 | $= 4$ |

The Welford update at fraction 3 is therefore $\mu' = (2\,\mu_{\text{start}} + o')/3$, treating `initial_belief_mu` as the result of two treatment-fraction observations. The information from $o_0$ is already encoded in `initial_belief_mu`'s value and is not double-counted.

## 6.2 Grid projection after the update

After computing continuous $\mu'$ and $\sigma'$, the values are clipped to the grid bounds and snapped to the nearest grid point. This is handled by `_hypothetical_belief_grid_indices` (scalar path) and the inline vectorised block inside `_bellman_expectation_full_grid` (full belief-grid path), both in `belief_model.py`:

1. Clip $\mu'$ to $[\mu_{\min}, \mu_{\max}] = [0, 30]$ cc, then find the nearest index in `_MU_GRID`.
2. Clip $\sigma'$ to $[\sigma_{\min}, \sigma_{\max}] = [0.05, 4.5]$ cc (enforcing the positive floor), then find the nearest index in `_SIGMA_GRID`.

The resulting pair of integers $(i', s')$ is the quantised updated belief.

### 6.3 Scalar belief update: `_hypothetical_belief_grid_indices`

This function, in `belief_model.py`, performs the Welford update for a *single* starting belief $(\mu, \sigma)$ over all $J$ overlap branches simultaneously. `volume_space` is an array of length $J$ playing the role of $o'$, so all $J$ updates are computed in parallel using NumPy vectorisation:

```
def _hypothetical_belief_grid_indices(mu, sigma, volume_space, observation_count):
    """For each hypothetical next overlap in volume_space, compute where the updated
    belief (mu, sigma) would land on the (_MU_GRID, _SIGMA_GRID) grid.

    Returns next_mi, next_si: (N_overlap,) integer index arrays into _MU_GRID and
    _SIGMA_GRID, used to look up future values in the DP values array.
    Uses Welford's online algorithm to update the running mean and variance.
    """
    mu_prime = (observation_count * mu + volume_space) / (observation_count + 1)
    sigma2_prime = (observation_count * sigma ** 2
                    + (volume_space - mu) * (volume_space - mu_prime)) / (observation_count + 1
    sigma_prime = np.sqrt(np.maximum(sigma2_prime, _SIGMA_MIN ** 2))
    mu_prime    = np.clip(mu_prime,    _MU_GRID[0],    _MU_GRID[-1])
    sigma_prime = np.clip(sigma_prime, _SIGMA_GRID[0], _SIGMA_GRID[-1])
    next_mu_index    = nearest_idx(mu_prime,    _MU_GRID)
    next_sigma_index = nearest_idx(sigma_prime, _SIGMA_GRID)
    return next_mu_index, next_sigma_index
```

The result is two integer arrays `next_mu_index` and `next_sigma_index`, each of length $J$. Element $j$ of each array gives the belief grid index after hypothetically observing branch $j$.

### 6.4 Expected future value for a scalar belief: `_bellman_expectation`

This function is in `belief_model.py`.

**What we want.** For a fixed starting belief $(\mu, \sigma)$ and a fixed accumulated dose state $D$, we want the expected future value:

$$E[V \mid D, \mu, \sigma] = \sum_{j=0}^{J-1} p_j^{(\mu,\sigma)}\, V_{\text{next}}[D,\ j,\ \texttt{next_mu_index}[j],\ \texttt{next_sigma_index}[j]]\,, \tag{13}$$

where $p_j^{(\mu,\sigma)}$ is the probability of branch $j$ given the current belief (a 1-D slice from `current_overlap_probs`), and $V_{\text{next}}[D, j, i', s']$ is the value stored in the previously-computed table at dose state $D$, overlap branch $j$, and updated belief $(i', s')$.

**Why fancy indexing is needed, and what `np.arange` does here.** Each branch $j$ points to a *different* belief index pair (`next_mu_index`$[j]$, `next_sigma_index`$[j]$). A plain slice like `values_prev[:, :, i', s']` would fix a single belief — not what we want.

The expression `values_prev[:, np.arange(J), next_mu_index, next_sigma_index]` uses advanced indexing across four axes simultaneously. `np.arange(J)` generates $[0, 1, 2, \ldots, J-1]$ — it is used to *pair each branch $j$ with itself* in the overlap axis, so that index $j$ in the overlap dimension always lines up with index $j$ in `next_mu_index` and `next_sigma_index`. Without `np.arange(J)`, NumPy would not know which overlap bin to pair with which belief index. The result is: for each $j$, take `values_prev[:, j, next_mu_index[j], next_sigma_index[j]]` — a different belief position for each branch.

```
def _bellman_expectation(values_prev, volume_space, p_branch, mu, sigma, observation_count):
    """Bellman expectation over all hypothetical next overlaps for a single belief (mu, sigma)

    Sums future values weighted by branch probabilities:
        sum_j p_branch[j] * values_prev[d, j, next_mi[j], next_si[j]]

    values_prev: (N_dose, N_overlap, N_mu, N_sigma)
    p_branch:    (N_overlap,) probability of each next overlap under current belief
    Returns:     (N_dose,) expected future value for each accumulated dose state
    """
    next_mi, next_si = _hypothetical_belief_grid_indices(mu, sigma, volume_space, observation_c
    branch_vals = values_prev[
        :, np.arange(len(volume_space)), next_mi, next_si
    ]  # shape: (N_dose, N_overlap)
    return (branch_vals * p_branch[None, :]).sum(axis=1)
```

After gathering, `branch_vals` has shape $(N_D, J)$: for each dose state $D$ and each branch $j$, the value at the updated belief. Multiplying by `p_branch[None, :]` (broadcast to $(N_D, J)$) and summing over $j$ gives the expected value as a $(N_D,)$ vector.

This scalar version is used only for the actual fraction's decision, where (`initial_belief_mu`, `initial_belief_` is a single point and `observation_count` is the current fraction index. Hypothetical future fractions use the fully vectorised version described in Section 7.8.

# 7 The Bellman Recursion: `adaptive_fractionation_core`

This function is the heart of the algorithm. It takes the current treatment state and returns the optimal dose and supporting value/policy tables.

## 7.1 Two key arrays: `dose_space` and `action_space`

Two arrays with similar names appear throughout the solver and are easy to confuse. It is worth defining them clearly before anything else.

`dose_space` is the grid of possible *accumulated* dose values $D$ — the total Gy already delivered to the tumour at the start of a fraction. It runs from the smallest reachable accumulated dose up to $D_{\text{goal}}$, plus one extra sentinel value $D_{\text{goal}} + 0.05$ (see below). The first axis of the value table `values` is indexed by `dose_space`.

`action_space` is the grid of possible *actions* — the doses the algorithm can choose to deliver in a single fraction. It runs from $d_{\min}$ to $d_{\max}$ in steps of $\delta_d$. With defaults this is a 9-point grid from 6.0 Gy to 10.0 Gy in 0.5 Gy steps.

In the code:

```
dose_space = np.arange(min_accumulated_dose_after_fraction, prescribed_dose, dose_steps)
overdose_sentinel = prescribed_dose + 0.05
dose_space = np.concatenate((dose_space, [prescribed_dose, overdose_sentinel]))

action_space = np.arange(min_dose, max_dose + 0.01, dose_steps)
```

**The** `overdose_sentinel.` `overdose_sentinel = prescribed_dose + 0.05` is a *sentinel value*: a deliberately out-of-range number used to mark overdose states. It is never a real accumulated dose; it is appended to `dose_space` so that any action which would push the total above $D_{\text{goal}}$ maps to this special entry. The terminal case assigns a $-10^{12}$ penalty to it, so the optimiser always avoids it. The value $+0.05$ is chosen to be small enough not to interfere with real doses but large enough to be distinguishable from $D_{\text{goal}}$ after floating-point arithmetic. Concretely, it must satisfy two constraints: (1) it must be less than half a dose step ($\delta_d/2 = 0.25$ Gy) so it cannot be confused with a legitimate accumulated dose; and (2) it must exceed the floating-point rounding error on $D_{\text{goal}}$ (of order $10^{-14}$ Gy for the 40 Gy default). The value 0.05 Gy satisfies both with wide margins and is otherwise arbitrary within this safe range.

## 7.2 State representation and array layout

The DP value table `values` has shape $(n_s, N_D, J, N_\mu, N_\sigma)$, where:

- $n_s = F - f$: number of future fractions (depth of the backward pass);
- $N_D$: number of accumulated-dose grid points;
- $J = 441$: number of overlap grid points;
- $N_\mu = 150$, $N_\sigma = 11$: belief grid sizes.

So `values[k, d, j, i, s]` is the optimal expected value from fraction $(F - k)$ onwards for dose index $d$, overlap index $j$, and belief-grid indices $(i, s)$. Concretely, these correspond to accumulated dose `dose_space[d]`, overlap `_VOLUME_SPACE[j]`, and belief (`_MU_GRID`$[i]$, `_SIGMA_GRID`$[s]$).

**Note 7.1.** The backward-pass loop (`_build_dp_context`) runs for $n_s$ iterations ($i = 0$ through $i = n_s - 1$), covering only future fractions $f + 1$ through $F$. It writes into `values[0]` through `values[n_s-1]`. The actual current fraction $f$ is resolved separately by `_resolve_current_fraction`, which reads `values[-1]` (the table for fraction $f + 1$) and performs a single Bellman step at the true starting belief to produce `recommended_dose`.

The axis ordering `[k, d, j, i, s]` places the belief dimensions ($i$, $s$) at the end of the value table, where NumPy's row-major layout makes them most cache-friendly for the inner-loop fancy indexing in Step 3 of Section 7.8. Note that the action dimension does not appear in the value table at all; it is exposed by transposing a separate buffer `future_values_masked_action_last` before the Numba kernel call in Step 5 (see that section for details).

## 7.3 Action set

The set of doses the algorithm can choose at fraction $t$ is:

$$\mathcal{A}(D) = \{d_{\min},\ d_{\min} + \delta_d,\ \ldots,\ d_{\max}\} \cap \{d : d \leq D_{\text{goal}} - D\}. \tag{14}$$

The constraint $d \leq D_{\text{goal}} - D$ prevents immediate overdose. Discretised in the code:

```
action_space = np.arange(min_dose, max_dose + 0.01, dose_steps)
```

With defaults this gives 9 actions: 6.0, 6.5, 7.0, …, 10.0 Gy.

## 7.4 Hard feasibility guards

Before entering the DP loop, two checks determine whether the prescription is even achievable. If there are $L = F - f + 1$ fractions remaining:

- If $D_{\text{goal}} - D < L \cdot d_{\min}$: even delivering $d_{\min}$ every remaining fraction gives less than $D_{\text{goal}}$. Prescription is impossible; return $d_{\min}$ as the forced action.
- If $D_{\text{goal}} - D > L \cdot d_{\max}$: even delivering $d_{\max}$ every remaining fraction falls short of $D_{\text{goal}}$. Prescription is impossible; return $d_{\max}$.

```
if remaining_ptv_dose < remaining_fractions * min_dose:
    recommended_dose = min_dose
    ...
elif remaining_ptv_dose > remaining_fractions * max_dose:
    recommended_dose = max_dose
    ...
```

In both cases, all value table entries are set to $-10^{12}$ to signal that no valid DP solution exists.

### 7.5 Initialisation from observed data

```
# In _resolve_current_fraction (core_adaptfx.py):
all_volumes = np.append(prior_volumes, observed_overlap)
initial_belief_mu    = all_volumes.mean()
initial_belief_sigma = std_calc(all_volumes, alpha, beta)
current_overlap_probs = current_belief_probdist(initial_belief_mu, initial_belief_sigma)
```

`observed_overlap` is the overlap measured for the current fraction $f$, extracted by the caller as `volumes[-1]`. It is the true value the algorithm must act on; it is used later when computing the actual penalty `penalty_calc_single(..., observed_overlap)` and when determining the specific recommended dose.

`initial_belief_mu` is the sample mean of all $f$ observed overlaps (including the planning scan). `initial_belief_sigma` is the MAP estimate from Section 4.1. `current_overlap_probs` is the probability vector over the 441-point overlap grid under $\mathcal{N}(\texttt{initial_belief_mu}, \texttt{initial_belief_sigma}^2)$, computed by `current_belief_probdist` (in `belief_model.py`), which wraps `probdist` without tail-folding.

**Code organisation note.** The solver is split into two functions: `_build_dp_context` runs the backward DP sweep over all *future* fractions (Cases 1 and 2 below) and returns a reusable context dict; `_resolve_current_fraction` consumes that context to determine the dose for the *actual* current fraction (Cases 3–5 below). This split was introduced so that `precompute_plan` can call `_build_dp_context` *once* and then call `_resolve_current_fraction` cheaply for each candidate overlap value, instead of rebuilding the full DP table from scratch for every candidate.

### 7.6 The DP backward-pass loop

The loop runs from `i=0` (last fraction, $F$) to `i=remaining_fractions - 1` (the current fraction $f$), filling `values[i]` at each step using `values[i-1]` (which was just computed):

```
for i, fraction_number in enumerate(
        np.arange(number_of_fractions, fraction_index_today - 1, -1)):
    observation_count = int(fraction_number)
    ...
```

`np.arange(F, f-1, -1)` generates $[F, F-1, \ldots, f]$ where $f$ is `fraction_index_today`. `enumerate(...)` pairs each element with a counter starting from 0, so `i` counts up from 0 while `fraction_number` counts down from $F$ to `fraction_index_today` in lockstep.

The cast `observation_count = int(fraction_number)` converts the numpy integer (`int64` dtype, since `np.arange` receives integer arguments) into a plain Python integer. The arithmetic expressions that follow work either way, but the explicit cast makes the intent clear: `observation_count` is always a whole-number observation count.

The backward-pass loop (in `_build_dp_context`) covers only the *future* fractions $f+1$ through $F$; the actual current fraction $f$ is handled by `_resolve_current_fraction` afterwards. Within the backward loop there are two cases:

| Code condition | i | Role | Ex: F=5, f=2 |
|---|---|---|---|
| `i == 0` | 0 | Terminal fraction ($F$) | $i = 0$ (fraction 5) |
| `i $>$ 0` | $1 \ldots n_s - 1$ | Hypothetical fractions | $i = 1$ (frac 4), $i = 2$ (frac 3) |

After the backward-pass loop, `_resolve_current_fraction` handles the current fraction $f$ itself (Cases 3–5 below).

**Example 7.1** (Five-fraction treatment, planning at fraction 2)**.** $F = 5$, $f = 2$. The backward sweep ($n_s - 1 = 2$ future hypothetical fractions) produces:

| i | `fraction_number` | **Role** |
|---|---|---|
| 0 | 5 | Terminal: forces $d_5 = D_{\text{goal}} - D$ |
| 1 | 4 | Hypothetical fraction 4: full DP update |
| 2 | 3 | Hypothetical fraction 3: full DP update |

Then `_resolve_current_fraction` runs the actual-fraction step for $f = 2$, producing `recommended_dose` for fraction 2 given the three observed overlaps $o_0, o_1, o_2$.

## 7.7 Case 1: Terminal fraction (`i == 0`)

At the last fraction there is no more future to plan for. The action is uniquely determined: deliver exactly the remaining dose $D_{\text{goal}} - D$, clamped to $[d_{\min}, d_{\max}]$. The terminal value is the negative penalty of this forced action, with a hard $-10^{12}$ sentinel if rounding means the total ends up above or below $D_{\text{goal}}$.

**Why clamp to $[d_{\min}, d_{\max}]$?** If $D_{\text{goal}} - D$ falls outside the clinically allowed range, the prescription cannot be met exactly. The clamp returns the closest feasible dose; the sentinels then record the infeasibility.

```
# --- Terminal fraction (i == 0): no choice | deliver exactly the remaining dose ---
best_actions = np.clip(prescribed_dose - dose_space, min_dose, max_dose)
future_accumulated_dose = dose_space + best_actions

terminal_oar_penalty = penalty_calc_single(best_actions[:, None], min_dose, _VOLUME_SPACE[None
# shape: (N_dose, N_overlap)

underdose_penalty = np.zeros(future_accumulated_dose.shape)
overdose_penalty  = np.zeros(future_accumulated_dose.shape)
underdose_penalty[np.round(future_accumulated_dose, 2) < prescribed_dose] = -1e12
overdose_penalty[np.round(future_accumulated_dose, 2) > prescribed_dose]  = -1e12

terminal_state_value = (-terminal_oar_penalty + underdose_penalty[:, None] + overdose_penalty[
values[i]   = terminal_state_value[:, :, None, None]
policies[i] = best_actions[:, None, None, None] * np.ones((N_dose, N_overlap, N_mu, N_sigma))
```

**Why rounding?** Floating-point arithmetic on dose steps (e.g. $0.5 \times 5$) can leave the total slightly above or below 40 due to machine epsilon. Rounding to 2 decimal places suppresses this noise.

**Why is the terminal value belief-independent?** The action at the last fraction is forced — deliver exactly what is remaining — regardless of what the running belief $(\mu, \sigma)$ says. So the value and policy do not vary with the belief. Broadcasting `terminal_state_value[:, :, None, None]` expands the $(N_D, J)$ array over the belief dimensions without copying data.

Note that `np.clip(prescribed_dose - dose_space, min_dose, max_dose)` performs the clamping directly, replacing the earlier multi-step if-statement approach.

### 7.8 Case 2: Hypothetical future fractions (`i $>$ 0`)

This case fills `values[i]` for *every* combination of $(D, o, \mu, \sigma)$ at hypothetical future fraction `fraction_number`. It is the most computationally intensive path, executed for $n_s - 1$ fractions.

The computation is now split across two functions called from within the loop.

#### Steps 1+3 combined: Bellman expectation over the full belief grid

The expected future value for every (accumulated-dose, belief) combination is computed by `_bellman_expectation` from `belief_model.py`:

```
future_value_prob_full = _bellman_expectation_full_grid(values[i - 1], observation_count)
# shape: (N_dose, N_mu, N_sigma)
```

Internally this function:

1. Applies the Welford update vectorised over all $(N_\mu \times N_\sigma \times J)$ triples to obtain `next_mi` and `next_si` (belief grid indices after each hypothetical observation) — exactly the computation described in Section 2.5.

2. Flattens the belief index: $b' = i' \cdot N_\sigma + s'$, reshapes `values[i-1]` to `vals_flat` of shape $(N_D, J, N_\mu N_\sigma)$.

3. Accumulates the probability-weighted sum in a $j$-loop (one slice at a time) to avoid materialising the full $(N_D, N_\mu, N_\sigma, J)$ intermediate:

```
for j in range(N_overlap):
    future_value_prob_full += (
        vals_flat[:, j, :][:, next_b[:, :, j]]  # (N_dose, N_mu, N_sigma)
        * _P_BELIEF[None, :, :, j]               # (1, N_mu, N_sigma)
    )
```

#### Step 4: Look up future values at action-induced doses

Because `action_space` and `dose_space` are constructed with the same step size and starting alignment, every future accumulated dose $D_d + a_k$ lands *exactly* on a `dose_space` grid point. The code verifies this alignment via an assertion and then uses a direct `searchsorted` lookup (no interpolation required):

```
future_doses = dose_space[:, None] + action_space[None, :]
query_doses  = future_doses.ravel()
dose_indices = np.clip(
    np.searchsorted(dose_space, np.round(query_doses, decimals=10), side='left'),
    0, N_dose - 1
)
future_values_full = future_value_prob_full[dose_indices].reshape(
    N_dose, len(action_space), N_mu, N_sigma
)
# shape: (N_dose, N_action, N_mu, N_sigma)
```

The `np.round(..., decimals=10)` suppresses sub-epsilon floating-point drift from `np.arange` before the `searchsorted` call. After this step, `future_values_full[d, a, i, s]` is the expected future value after being in dose state $d$, taking action $a$, and holding belief $(i, s)$.

### Valid actions mask

Before the argmax step, a valid-actions mask is computed to prevent overshooting the prescription:

```
min_dose_reserved_for_future = (number_of_fractions - fraction_number) * min_dose
max_allowed_actions = np.minimum(action_space[-1],
    prescribed_dose - dose_space - min_dose_reserved_for_future)
max_action_indices = np.abs(action_space[None, :] - max_allowed_actions[:, None]).argmin(axis=
max_action_indices = np.where(max_action_indices == 0, 1, max_action_indices)
valid_actions = (np.arange(action_space.size)[None, :] <= max_action_indices[:, None])
# shape: (N_dose, N_action)
```

Invalid actions (those that would overshoot $D_{\text{goal}}$ from some dose states) are masked to `min_float` so `argmax` never selects them.

### Step 5: Value and policy update via Numba kernel

The Bellman equation for this step is:

$$V(d, j, i, s) = \max_{a \in \mathcal{A}(d)} \big[ -c(a, o_j) + \texttt{future_values_full}[d, a, i, s] \big]. \tag{15}$$

The masking and argmax are handled by the Numba-JIT kernel `_fill_values_policies` in `core_adaptfx.py`. Before calling it, the code prepares inputs:

```
future_values_masked = np.where(
    valid_actions[:, :, None, None], future_values_full, min_float
)
# Transpose so action axis is last for cache-friendly argmax:
future_values_masked_action_last = future_values_masked.transpose(0, 2, 3, 1).copy()
# shape: (N_dose, N_mu, N_sigma, N_action), contiguous
```

`np.finfo(np.float64).min` (`min_float`) is the most negative finite 64-bit float ($\approx -1.8 \times 10^{308}$). It is used instead of `-np.inf` because `argmax` can return `nan` in edge cases when `-np.inf` is present.

A flat-index base is precomputed to allow the Numba kernel to gather the winning value without `take_along_axis` (which allocates a temporary):

```
flat_index_base = (
    np.arange(N_dose)[:, None, None] * N_mu * N_sigma
    + np.arange(N_mu)[None, :, None] * N_sigma
    + np.arange(N_sigma)[None, None, :]
) * len(action_space)
# shape: (N_dose, N_mu, N_sigma)
```

The kernel then iterates over all $J = 441$ overlap bins in parallel (via `prange`), subtracting the per-overlap penalty from `future_values_masked_action_last` and recording the best action and value:

```
_fill_values_policies(
    future_values_masked_action_last, overlap_penalty,
    flat_index_base, action_space, values_state, policies_state
)
values[i]   = values_state
policies[i] = policies_state
```

**Why Numba?** The $j$-loop over 441 overlap bins is the performance bottleneck. Parallelising it with `prange` (Numba's parallel range) gives a roughly $N_{\text{cores}}$-fold speedup. The kernel subtracts `overlap_penalty[j]` from the entire $(N_D, N_\mu, N_\sigma, N_a)$ buffer, finds the argmax, and gathers the winning value using the precomputed flat index — all without heap allocation inside the loop.

`flat_index_base` — **row-major index arithmetic.** NumPy stores multi-dimensional arrays in row-major (C) order: the last axis varies fastest in memory. For a 4-D array of shape $(N_D, N_\mu, N_\sigma, N_a)$, the element at position $(d, i, s, a)$ sits at flat memory offset:

$$\text{flat} = d \cdot N_\mu N_\sigma N_a + i \cdot N_\sigma N_a + s \cdot N_a + a.$$

`flat_index_base[d, i, s]` precomputes the first three terms $(d \cdot N_\mu N_\sigma + i \cdot N_\sigma + s) \cdot N_a$ for every $(d, i, s)$ triple, broadcast together via the three `[:, None, None]` / `[None, :, None]` / `[None, None, :]` shapes (Section 2.5). Adding the best action index from `argmax` gives the exact flat offset of the winning value — no temporary array created.

### 7.9 Case 3: Actual fraction — first fraction ($f = 1$)

When planning fraction 1, all future fractions have been filled as hypothetical (Case 2), and we now compute the recommendation for the actual fraction. Since $f = 1$, there is no accumulated dose ($D_1 = 0$), so the dose-space axis is free. This case is reached when `fraction_index_today == 1`.

The key difference from Case 2 is that we use a *single* starting belief (`initial_belief_mu`, `initial_belief_sig` — the initial belief from real observations — rather than computing over the full belief grid. We call `_bellman_expectation` which handles the scalar case:

```
# fraction_index_today == 1: actual first fraction (in _resolve_current_fraction)
immediate_cost = penalty_calc_single(action_space, min_dose, observed_overlap)

future_value_prob = _bellman_expectation(
    values[-1], _VOLUME_SPACE, current_overlap_probs,
    initial_belief_mu, initial_belief_sigma, observation_count
)
# Direct lookup: action_space and dose_space are perfectly aligned at fraction 1
action_dose_indices = np.clip(
    np.searchsorted(dose_space, np.round(action_space, decimals=10), side='left'),
    0, N_dose - 1
)
future_values = future_value_prob[action_dose_indices]

state_values_full_grid = -overlap_penalty + future_values
```

```
current_fraction_policy = action_space[state_values_full_grid.argmax(axis=1)]

actual_value   = -immediate_cost + future_values
recommended_dose = action_space[actual_value.argmax()]
```

`state_values_full_grid` has shape $(J, N_a)$: for each possible overlap $j$ and each action $a$, the total value (immediate reward plus expected future). `current_fraction_policy[j]` is the best action given overlap $j$ (the full policy curve for this fraction). `recommended_dose` is the recommendation for the *observed* overlap `observed_overlap`.

At fraction 1, `action_space` and `dose_space` are built with the same step size starting from the same minimum, so every action $a_k$ corresponds exactly to a dose-space grid point. A direct `searchsorted` lookup (no interpolation) is therefore correct here.

### 7.10 Case 4: Actual fraction — intermediate ($f < F$, $f > 1$)

For fractions other than the first, the accumulated dose $D_f > 0$, which restricts the action set further. Feasibility clipping is now performed inline (Section 4.5) rather than via a separate helper:

```
# Middle fraction (not first, not last): in _resolve_current_fraction
min_dose_reserved_for_future = (number_of_fractions - fraction_index_today) * min_dose
max_allowed_action = np.minimum(action_space[-1],
    prescribed_dose - accumulated_dose - min_dose_reserved_for_future)
max_action_index = np.abs(action_space - max_allowed_action).argmin()
max_action_index = max(max_action_index, 1)
feasible_actions = action_space[:max_action_index + 1]

feasible_overlap_penalty = penalty_calc_matrix(feasible_actions, _VOLUME_SPACE, min_dose)
immediate_cost  = penalty_calc_single(feasible_actions, min_dose, observed_overlap)

future_accumulated_dose = accumulated_dose + feasible_actions
future_accumulated_dose[future_accumulated_dose > prescribed_dose] = overdose_sentinel
overdose_penalties = np.zeros(future_accumulated_dose.shape)
overdose_penalties[future_accumulated_dose > prescribed_dose] = -1e12

future_value_prob = _bellman_expectation(
    values[-1], _VOLUME_SPACE, current_overlap_probs,
    initial_belief_mu, initial_belief_sigma, observation_count
)
future_values = linear_interp(dose_space, future_value_prob, future_accumulated_dose)

state_values_full_grid = -feasible_overlap_penalty + future_values + overdose_penalties
current_fraction_policy = feasible_actions[state_values_full_grid.argmax(axis=1)]

actual_value     = -immediate_cost + future_values + overdose_penalties
recommended_dose = feasible_actions[actual_value.argmax()]
```

The logic is identical to Case 3 except that `feasible_actions` is shorter (high actions that would overshoot are excluded), and `accumulated_dose` shifts the future-dose axis. Note that `linear_interp` IS still used here (unlike fraction 1) because `accumulated_dose` is an external caller input that may not lie exactly on the grid.

### 7.11 Case 5: Actual fraction is the last fraction ($f = F$)

No DP is needed: deliver exactly what is left. This case is reached when `fraction_index_today == number_of_fractions`.

```
elif fraction_index_today == number_of_fractions:
    recommended_dose = np.clip(remaining_ptv_dose, min_dose, max_dose)
    actual_value = np.zeros(1)
```

`np.clip` clamps `remaining_ptv_dose` to $[d_{\min}, d_{\max}]$.

`actual_value = np.zeros(1)` represents a value of zero for the last fraction: there is no future left to optimise, so the value is defined to be zero (the penalty was already paid in the terminal case calculation).

### 7.12 Return values

After the loop, results are rounded and returned:

```
physical_dose = np.round(recommended_dose, 2)
penalty_added = penalty_calc_single(physical_dose, min_dose, observed_overlap)
optimal_state_value = np.max(actual_value)
return [
    policies,              # (n_s, N_D, J, N_mu, N_sigma) -- full policy table
    current_fraction_policy,  # (J,) -- optimal dose for every overlap value
    volume_space,          # (J,) -- the overlap grid used
    physical_dose,         # float -- recommended dose for this fraction
    penalty_added,         # float -- OAR cost of the recommended dose
    values,                # (n_s, N_D, J, N_mu, N_sigma) -- full value table
    dose_space,            # (N_D,) -- accumulated dose grid
    current_overlap_probs, # (J,) -- current belief probabilities
    optimal_state_value,   # float -- best total expected OAR cost from this fraction onward
]
```

Most callers only use index 3 (the recommended physical dose). The full tables are available for visualisation and debugging.

**Note 7.2.** `optimal_state_value` is `np.max(actual_value)`, the best total expected OAR cost from this fraction to end of treatment. It is negative (less negative = better; a value of 0 at the last fraction is correct since no future fractions remain). Most callers only use index 3, the recommended physical dose.

# 8 Public API Functions

## 8.1 `adaptfx_full`: replay a complete treatment course

**Purpose.** Given all $F$ overlap volumes for a complete treatment (known in retrospect), simulate what the algorithm would have recommended at each fraction. Returns the full dose sequence and the total penalty.

**Why this is the primary benchmarking entry point.** In the 58-patient benchmark cohort, the true overlaps for each fraction are known. `adaptfx_full` replays the algorithm on each patient's real trajectory — as if we were planning fraction by fraction with only the overlaps seen so far available — and compares the resulting total penalty against both the old solver and the theoretical ideal upper bound.

**Note 8.1** (Sign convention for `total_penalty`)**.** Despite its name, `total_penalty` is computed using `-=` applied to non-negative per-fraction penalty values, so the returned value is $\leq 0$. It is the accumulated *reward* (negative penalty) over the treatment course, not the raw total penalty. A value closer to zero means a better outcome (lower total OAR cost); in benchmarking, higher (less negative) is better.

```
def adaptfx_full(volumes, number_of_fractions=DEFAULT_NUMBER_OF_FRACTIONS,
                 min_dose=DEFAULT_MIN_DOSE, max_dose=DEFAULT_MAX_DOSE,
                 mean_dose=DEFAULT_MEAN_DOSE, ...):
    physical_doses    = np.zeros(number_of_fractions)
    accumulated_doses = np.zeros(number_of_fractions)

    for index, frac in enumerate(range(1, number_of_fractions + 1)):
        if frac != number_of_fractions:
            # At fraction frac, overlaps 0..frac are known
            # (planning scan o_0 plus treatment fractions o_1..o_frac).
            # Slice: volumes[:-F+frac] gives the first frac+1 elements.
            physical_dose = adaptive_fractionation_core(
                fraction_index_today=frac,
                volumes=np.array(volumes[:-number_of_fractions + frac]),
                accumulated_dose=accumulated_doses[index],
                ...
            )[3]   # index 3 is the recommended dose
            accumulated_doses[index + 1] = accumulated_doses[index] + physical_dose
        else:
            # Last fraction: use all observed overlaps
            physical_dose = adaptive_fractionation_core(
                fraction_index_today=frac,
                volumes=np.array(volumes),
                accumulated_dose=accumulated_doses[index],
                ...
            )[3]
        physical_doses[index] = physical_dose

    total_penalty = 0
```

```
    for index, dose in enumerate(physical_doses):
        total_penalty -= penalty_calc_single(
            dose, min_dose, volumes[-number_of_fractions + index]
        )
    return physical_doses, accumulated_doses, total_penalty
```

**Slice logic.** At fraction $f$, the slice `volumes[:-F+f]` gives `volumes[0:f+1]`, i.e. the first $f+1$ overlaps ($o_0$ through $o_f$: the planning scan plus all $f$ delivered treatment fractions, including the current one). At $f = F$ the full array is used directly.

### 8.2 `precompute_plan`: build a dose lookup table

**Purpose.** Before the overlap for the next fraction is measured, precompute the dose the algorithm would recommend for each possible overlap value. The result is a lookup table; once the real overlap is measured, the dose can be read off immediately without waiting for the DP.

**Scanning loop.** The function scans candidate overlaps from 0 cc upward in 0.1 cc steps. For each candidate, it injects that value as the last element of the observed overlap array and calls `adaptive_fractionation_core` to get the recommendation.

```
def precompute_plan(fraction_index_today, volumes, accumulated_dose, ...):
    volumes = np.asarray(volumes, dtype=float)
    std = std_calc(volumes, alpha, beta)
    distribution_params = (volumes.mean(), std)
    volume_space = get_state_space(distribution_params)
    distribution_max = (6.5 if volume_space.max() < 6.5
                        else volume_space.max())
    min_dose_deliverable = min_dose_to_deliver(
        accumulated_dose=accumulated_dose,
        fractions_left=number_of_fractions - fraction_index_today + 1,
        prescribed_dose=mean_dose * number_of_fractions,
        min_dose=min_dose, max_dose=max_dose
    )

    # Run the expensive DP backward sweep ONCE | independent of which
    # overlap will be observed, so the result is reused for every candidate.
    ctx = _build_dp_context(
        fraction_index_today, number_of_fractions, accumulated_dose,
        min_dose, max_dose, mean_dose, dose_steps
    )

    volumes_to_check = [0.0]
    predicted_policies = []

    while True:
        volume = volumes_to_check[-1]
        # Cheap per-candidate lookup: only the current-fraction step is re-run.
        recommended_dose, _, _, _ = _resolve_current_fraction(
```

```
        ctx, fraction_index_today, number_of_fractions, volume, volumes, alpha, beta
    )
    physical_dose = np.round(recommended_dose, 2)
    predicted_policies.append(physical_dose)

    covered_distribution_range = volume >= distribution_max
    reached_minimum_policy = physical_dose <= min_dose_deliverable
    if covered_distribution_range and reached_minimum_policy:
        break
    volumes_to_check.append(np.round(volume + 0.1, 10))

...
return volume_x_dose, volumes_to_check, predicted_policies
```

**Stopping criterion.** Scanning stops when *both* conditions hold:

1. The scanned overlap exceeds `distribution_max`, which is the 99.9th percentile of the current belief $\mathcal{N}(\mu, \sigma^2)$, floored at **6.5 cc**. Beyond this point, the probability of a real overlap landing there is negligible.

2. The recommended dose has dropped to the minimum feasible dose. Once the dose saturates at the minimum, further scanning cannot produce a different recommendation.

**Why 6.5 cc as the floor?** `get_state_space` returns the 0.1th–99.9th percentile interval of $\mathcal{N}(\mu, \sigma^2)$. For patients with small mean overlap and $\sigma$ close to the population prior mean ($\approx$ 0.78 cc), the 99.9th percentile can fall well below 2 cc—far below the volume at which the policy drops to the minimum dose. Without a floor, the scan would stop too early for such patients and the returned lookup table would be incomplete: high-overlap entries would not be computed at all.

6.5 cc is an empirically-derived safety margin: across the full 58-patient ACTION cohort with default dose parameters ($d_{\min} = 6$ Gy, $d_{\max} \leq 10$ Gy), the optimal policy is observed to saturate at the minimum dose for all patients before the scanned volume reaches 6.5 cc. It is therefore the smallest floor value that makes condition 1 a safe stopping guard for every patient in the dataset. It is *not* a formal clinical protocol threshold.

**Backward-sweep caching.** The DP backward sweep (`_build_dp_context`) is independent of which overlap will be observed at the current fraction, so it is called *once* before the scanning loop. Each candidate overlap in the loop then only requires the cheap `_resolve_current_fraction` step (Bellman expectation at the actual starting belief), which is $O(N_{\text{overlap}})$ rather than $O(n_s \times N_D \times N_\mu \times N_\sigma \times N_{\text{overlap}})$. This makes `precompute_plan` roughly $N_{\text{candidates}}\times$ faster than calling the full `adaptive_fractionation_core` once per candidate.

**Float-drift guard.** The step `np.round(volume + 0.1, 10)` prevents accumulated floating-point error from repeated additions of 0.1 (which is not exactly representable in binary). After many iterations, unguarded addition would drift from the true value.

# 9 End-to-End Call Flow

The following steps trace a complete planning call for a mid-treatment fraction ($1 < f < F$). Numbers in parentheses refer to the sections that describe each step.

1. User calls `adaptive_fractionation_core(...)` with `volumes = o_{0:f}` and accumulated dose $D_f$. Here $o_{0:f}$ denotes the planning scan overlap $o_0$ followed by the $f$ treatment-fraction overlaps $o_1, \ldots, o_f$; `volumes` has length $f + 1$.

2. **Backward sweep** — `_build_dp_context` (Section 7):

   (a) **Feasibility check** (Section 7.4): if $D_{\text{goal}} - D_f$ is outside the reachable range, return an infeasible context immediately.

   (b) **i=0** (Section 7.7): fill terminal values using forced action; broadcast over belief dims.

   (c) **i=1...** $n_s - 1$ (Section 7.8): for each hypothetical future fraction:

      i. `_bellman_expectation_full_grid`: vectorised Welford update over all $(i, s, j)$ triples, then probability-weighted accumulation → `future_value_prob_full`.

      ii. Direct `searchsorted` lookup onto action-induced doses.

      iii. Valid-actions mask computed inline.

      iv. Numba kernel `_fill_values_policies`: subtract per-overlap penalty and `argmax` to fill `values[i]`.

3. **Actual-fraction step** — `_resolve_current_fraction`:

   (a) Compute `initial_belief_mu` = mean($o_{0:f}$).

   (b) Compute `initial_belief_sigma` = `std_calc`($o_{0:f}, \alpha, \beta$) (MAP estimate with Gamma prior).

   (c) Compute `current_overlap_probs` = `current_belief_probdist`(`initial_belief_mu`, `initial_beli`
   (441-element probability vector over `_VOLUME_SPACE`).

   (d) `_bellman_expectation` at the single starting belief using `values[-1]` from the backward sweep.

   (e) `linear_interp` onto the clipped action array (middle fractions) or direct lookup (fraction 1).

   (f) `argmax` gives `recommended_dose`.

4. Round `recommended_dose` to 2 decimal places.

5. Return 9-element result list; index 3 is the recommended dose.

**Note 9.1** (Edge cases: $f = 1$ and $f = F$)**.** The walkthrough above covers a mid-treatment fraction ($1 < f < F$). Two edge cases differ:

- **First fraction ($f = 1$)**: No accumulated dose ($D_1 = 0$), so the action set is not clipped by previous deliveries. The backward pass runs states 0 through $n_s = F - 1$, ending at Case 3 (Section 7.9) instead of Case 4.

- **Last fraction ($f = F$)**: No DP is performed. The function detects that the current fraction is the final one and delivers the remaining dose directly (Case 5: "Actual fraction is the last fraction"). It then returns without computing or storing any value or policy tables.

# 10 Grid Sizes and Computational Cost

## 10.1 Grid dimensions at default settings

| Quantity | Symbol | Value |
|---|---|---|
| Mu grid points | $N_\mu$ | 150 |
| Sigma grid points | $N_\sigma$ | 11 |
| Overlap grid points | $J$ | 441 |
| Dose-state points | $N_D$ | $\approx (D_{\text{goal}} - D_f)/\delta_d + 2$ |
| Action points | $N_a$ | $(d_{\max} - d_{\min})/\delta_d + 1 = 9$ |
| Fractions remaining | $n_s$ | $F - f$ |

At fraction 1 with defaults: $N_D \approx 70$ (`np.arange(6.0,\,40.0,\,0.5)` gives 68 points, plus the `goal` and sentinel entries), $n_s = 4$.

## 10.2 Memory

The dominant memory cost comes from the value and policy tables, each allocated at full 5D shape:

$n_s \times N_D \times J \times N_\mu \times N_\sigma \times 8$ bytes $= 4 \times 70 \times 441 \times 150 \times 11 \times 8 \approx 1{,}630$ MB $\approx 1.63$ GB per array.

Both `values` and `policies` are allocated at this size, giving a combined DP footprint of approximately **3.26 GB** at fraction 1. The precomputed `_P_BELIEF` array adds a further 6 MB.

**Note 10.1.** The large memory footprint is a known cost of the 5D extension's full belief-state representation. It is the primary reason planning calls take on the order of tens of seconds.

**Note 10.2** (System requirements)**.** The 3.26 GB figure covers only the `values` and `policies` arrays. Additional RAM is consumed by the Python interpreter, OS, the 6 MB `_P_BELIEF` table, and temporary intermediate arrays created during the DP loop (e.g. Welford arrays of shape $150 \times 11 \times 441$). In practice, total resident memory at fraction 1 is approximately 4–5 GB.

Practical guidance for first-time setup:

- **16 GB RAM or more**: recommended; arrays fit comfortably with room for OS and intermediate buffers.
- **8 GB RAM**: marginal at fraction 1; the system will compete for the remaining ∼3–4 GB. Expect longer planning times and possible paging on some configurations.
- **Less than 8 GB RAM**: likely to cause out-of-memory errors or extreme slowness at fraction 1, which has the largest $n_s = F - 1$ backward-pass depth.

No formal minimum-RAM test has been performed; the figures above are derived from the memory accounting in Section 10.2.

## 10.3 Operation count

For each hypothetical future fraction, the dominant operations are:

- **Welford update** (Step 1): $O(N_\mu \cdot N_\sigma \cdot J)$ element-wise operations $= 150 \times 11 \times 441 \approx 0.73$ million.
- **Weighted sum** (Step 3): $O(J \cdot N_D \cdot N_\mu \cdot N_\sigma) = 441 \times 70 \times 150 \times 11 \approx 51$ million.
- **Value update** (Step 5): $O(J \cdot N_D \cdot N_\mu \cdot N_\sigma \cdot N_a) = 441 \times 70 \times 150 \times 11 \times 9 \approx 458$ million.

Summing over $n_s - 1 = 3$ hypothetical fractions: $\approx 3 \times 510 \approx 1{,}530$ million (1.5 billion) operations.

## 11 Key Design Decisions and Their Rationale

This section summarises the non-obvious design choices and explains why each was made. Understanding these decisions is important for anyone modifying the code.

**Fixed wide `_VOLUME_SPACE` (not patient-specific).** Section 5.3 explains in detail why using a patient-specific grid corrupts the DP for beliefs far from the observed range. The fixed wide grid is the correct fix: it ensures correct transitions for all $150 \times 11$ belief grid points, including extreme ones.

**Tail assignment in `_P_BELIEF`.** Without equations (9)–(10), rows of `_P_BELIEF` would sum to less than 1 for beliefs near the edges of the range. The weighted sum in Step 3 would then systematically underestimate the expected future value, biasing the policy toward lower doses.

**Non-uniform `_MU_GRID` with fine resolution near 0 cc.** A patient with $\mu = 0.37$ cc mapped to the grid point $\mu = 0$ would see all branches land at the same overlap bin (0 cc), giving zero penalty everywhere and making the DP assign arbitrary doses. Fine resolution in $[0, 1]$ cc prevents this.

**`_MU_GRID` extended to 30 cc.** Patients with observed overlaps $> 12$ cc exist in the benchmark. Clipping to 12 cc would underestimate their distribution and produce overly aggressive dosing for those patients.

**Population variance (`ddof=0`) throughout.** The published 2025 paper uses population variance. Switching to Bessel's correction (`ddof=1`) would give `nan` at $n = 1$ and would produce different behaviour at small $n$, breaking the regression baseline.

**MAP estimate for $\sigma_{\text{start}}$ via discrete argmax.** The analytical MAP formula involves solving a polynomial equation; the discrete grid scan is simpler, robust at $n = 1$ (where the data term vanishes and the prior dominates), and accurate to well within clinical precision.

**Precomputed `_P_BELIEF` at module load time.** The probability array depends only on fixed constants. Precomputing it once eliminates $N_\mu \times N_\sigma = 1{,}650$ batched CDF evaluations from every planning call. The array is computed by `_precompute_branch_probabilities()` in `belief_model.py`.

**$j$-loop with accumulation in `_bellman_expectation_full_grid` (instead of full materialisation).** The full intermediate array would exceed 600 MB. The $j$-loop keeps peak memory at $\approx 1.6$ MB per iteration (one $j$-slice), which fits in L3 cache and allows the loop to run at memory-bandwidth speed rather than main-memory speed.

**Transpose before the Numba kernel (`future_values_masked_action_last`).** The action axis is moved to the last position so the `argmax` inside `_fill_values_policies` scans a contiguous memory region — approximately 18% faster.

**Numba JIT kernel `_fill_values_policies` for the innermost $j$-loop.** The $j$-loop over 441 overlap bins (the Step 5 argmax) is the throughput bottleneck. Parallelising it with `prange` gives a multi-core speedup; the flat-index trick avoids heap allocation inside each iteration.

**`searchsorted` for `nearest_idx`.** Binary search ($O(n \log K)$) is $\approx 20\times$ faster than `argmin` ($O(nK)$) for the 150-point mu grid on large arrays. This function is called once per Step 1 of every hypothetical fraction, so the speedup is significant.

**Flat belief index arithmetic instead of `take_along_axis`.** `take_along_axis` allocates a temporary array on every call. Precomputing `flat_index_base` and using integer addition inside the Numba kernel avoids this allocation and saves one heap allocation per iteration.